

torch.linalg.householder_product#

[https://pytorch.org/docs/stable/generated/torch.linalg.householder_product.h](https://pytorch.org/docs/stable/generated/torch.linalg.householder_product.html)

Description:

Computes the first n columns of a product of Householder matrices. Let $K \in \mathbb{R}^{m \times n}$ or $\mathbb{C}^{m \times n}$, and let $A \in \mathbb{R}^{m \times n}$ or $\mathbb{C}^{m \times n}$ be a matrix with columns $a_i \in \mathbb{R}^m$ or $\mathbb{C}^m$ for $i=1, \dots, n$ with $m \geq n$. Denote by b_i the vector resulting from zeroing out the first $i-1$ components of a_i and setting to 1 the i -th. For a vector $\tau \in \mathbb{R}^n$ or $\mathbb{C}^n$ with $\|\tau\| \leq 1$, this function computes the first n columns of the matrix where I_m is the m -dimensional identity matrix and $b_i b_i^H$ is the conjugate transpose when b_i is complex, and the transpose when b_i is real-valued. The output matrix is the same size as the input matrix A . See Representation of Orthogonal or Unitary Matrices for further details. Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

Function Signature

```
torch.linalg.householder_product(A, tau, *, out=None) →  
Tensor#
```

Parameters

Keyword Arguments

Raises

Parameters

Keyword

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
>>> A = torch.randn(2, 2)
>>> h, tau = torch.geqrf(A)
>>> Q = torch.linalg.householder_product(h, tau)
>>> torch.dist(Q, torch.linalg.qr(A).Q)
tensor(0.)

>>> h = torch.randn(3, 2, 2, dtype=torch.complex128)
>>> tau = torch.randn(3, 1, dtype=torch.complex128)
>>> Q = torch.linalg.householder_product(h, tau)
>>> Q
tensor([[[ 1.8034+0.4184j,  0.2588-1.0174j],
          [-0.6853+0.7953j,  2.0790+0.5620j]],

        [[ 1.4581+1.6989j, -1.5360+0.1193j],
          [ 1.3877-0.6691j,  1.3512+1.3024j]],

        [[ 1.4766+0.5783j,  0.0361+0.6587j],
          [ 0.6396+0.1612j,  1.3693+0.4481j]]],
        dtype=torch.complex128)
```

Notes & Warnings

NOTE

See also `torch.geqrf()` can be used together with this function to form the Q from the `qr()` decomposition. `torch.ormqr()` is a related function that computes the matrix multiplication of a product of Householder matrices with another matrix. However, that function is not supported by autograd.

⚠ WARNING

Warning Gradient computations are only well-defined if $\tau_i \neq 1 \parallel a_i \parallel^2 \tau_i \neq \frac{1}{\parallel a_i \parallel^2} \tau_i = \parallel a_i \parallel^2$. If this condition is not met, no error will be thrown, but the gradient produced may contain NaN.

Detailed Documentation

Documentation

Computes the first n columns of a product of Householder matrices.

Let $K \in \mathbb{R}^m \times \mathbb{R}^n$ or $\mathbb{C}^m \times \mathbb{C}^n$, and let $A \in K^{m \times n}$. Let $a_i \in K^m$ be a matrix with columns $a_i \in K^m$ for $i=1, \dots, m$ with $m \geq n$. Denote by b_i the vector resulting from zeroing out the first $i-1$ components of a_i and setting to 1 the i -th. For a vector $\tau \in K^k$ with $k \leq n$, this function computes the first n columns of the matrix

where I_m is the m -dimensional identity matrix and bHb^{H} is the conjugate transpose when b is complex, and the transpose when b is real-valued. The output matrix is the same size as the input matrix A .

See Representation of Orthogonal or Unitary Matrices for further details.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

`torch.geqrf()` can be used together with this function to form the Q from the `qr()` decomposition.

`torch.ormqr()` is a related function that computes the matrix multiplication of a product of Householder matrices with another matrix. However, that function is not supported by autograd.

Gradient computations are only well-defined if $\tau_i \neq 1 / \|a_i\|_2$. If this condition is not met, no error will be thrown, but the gradient produced may contain NaN.

A (Tensor) – tensor of shape $(*, m, n)$ where $*$ is zero or more batch dimensions.

τ (Tensor) – tensor of shape $(*, k)$ where $*$ is zero or more batch dimensions.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

`RuntimeError` – if A doesn't satisfy the requirement $m \geq n$, or τ doesn't satisfy the requirement $n \geq k$.

